Exercise Sheet 04

The Roster

House rules.

- One file per exercise, in `my_work/`: `ex1_roster.py`, `ex2_rewards.py`, …
- Type the **Given** lines first. Then write the rest yourself. Try each step in the REPL before you put it in the file.
- Run it (`python my_work/ex1_roster.py`) and compare with the **expected output**, character by character. When Python prints a list or a dictionary itself, it uses single quotes. That's fine; don't fight it.
- Hand in: commit and push to `lesson-04`, then open a pull request. Tell Lars one thing a dictionary does better than a list.

Exercise 1 The roster

10 min · `ex1_roster.py`

> **RAVEN #401, from Quartermistress Pell:** *"Orrin says I've been keeping the roster in a text with slashes in it. He said it in the tone he uses for people who store dates as text. Apparently there's a proper thing. P."*

GIVEN

```
heroes = ["Bram Ironfoot", "Wren Hawkridge", "Tamsin Larkspur", "Osric Kettleby"]
```

(a) Print the first hero, the last hero, and how many there are.

(b) Elowen Fairweather has joined the guild. `append` her and print the whole list.

(c) Print whether `"Wren Hawkridge"` is in the list, and whether `"Dalvric"` is.

(d) Osric Kettleby has retired to run an inn. `remove` him and print the list again.

(e) Print a slice of the second and third heroes. Slicing works like it did on text.

EXPECTED OUTPUT

```
Bram Ironfoot
Osric Kettleby
4
['Bram Ironfoot', 'Wren Hawkridge', 'Tamsin Larkspur', 'Osric Kettleby', 'Elowen
Fairweather']
True
False
['Bram Ironfoot', 'Wren Hawkridge', 'Tamsin Larkspur', 'Elowen Fairweather']
['Wren Hawkridge', 'Tamsin Larkspur']
```

Hint: Position 0 is the first, −1 is the last. Same rule as for letters in a string: a list is a row of things, a string is a row of characters.

Exercise 2 A column of rewards

10 min · `ex2_rewards.py`

> **RAVEN #402, from Queen Aldara, at dusk:** *"SUM, MAX, MIN, AVERAGE, and the top three. The things I do in Excel while pretending to listen to the Privy Council. These are the rewards on this week's notice board. A."*

GIVEN

```
rewards = [320, 1240, 150, 610, 2850, 480, 270, 760]
```

(a) Print the total, then the highest and lowest on one line, then the average rounded to 2 decimals.

(b) Print the rewards sorted low to high, then the top three (sort the other way, slice the first three).

(c) Build a new list `big` of the rewards over 500: start with `[]`, loop, `append`. Print it, then "4 of 8 notices pay over 500 crowns" using `len` twice.

EXPECTED OUTPUT

```
6680
2850 150
835.0
[150, 270, 320, 480, 610, 760, 1240, 2850]
[2850, 1240, 760]
[1240, 610, 2850, 760]
4 of 8 notices pay over 500 crowns
```

Hint: `sorted(rewards)` gives a sorted copy and leaves the original alone. `rewards.sort()` sorts in place and returns nothing. Guess which one Dalvric used in a print.

Exercise 3 One hero, many fields

10 min · `ex3_hero_dict.py`

> **RAVEN #403, from Orrin of the Clockworks:** *"A hero isn't a list. A hero has a name, a class, a level. In Python that's a dictionary: labels on the left, values on the right. Like one row of a table, with the headers attached."*

GIVEN

```
hero = {
    "name": "Wren Hawkridge",
    "class": "Ranger",
    "level": 19,
    "quests": 48,
    "crowns": 21360,
}
```

(a) Print the name, then the crowns she earns per quest on average (just divide; no rounding).

(b) Add a new field `"pet"` with the value `"a hawk called Tuesday"`, change the level to 20 (she had a good week), and print the whole dictionary.

(c) Print `hero.get("dragons_slain", 0)`. There is no such field: `.get` gives you the fallback instead of crashing. Then print how many fields there are.

(d) Loop over the dictionary (`for key in hero:`) and print each key left-aligned in 12 characters followed by its value.

EXPECTED OUTPUT

```
Wren Hawkridge
445.0
{'name': 'Wren Hawkridge', 'class': 'Ranger', 'level': 20, 'quests': 48, 'crowns': 21360,
'pet': 'a hawk called Tuesday'}
0
6
name        Wren Hawkridge
class       Ranger
level       20
quests      48
crowns      21360
pet         a hawk called Tuesday
```

Exercise 4 A list of dictionaries

15 min · `ex4_party.py`

> **RAVEN #404, from Quartermistress Pell:** *"So a dictionary is a row and a list is a column. Can I have a table? With the Brackenford party in it? Each with their share of the loot?"*

GIVEN

```
party = [
    {"name": "Bram Ironfoot", "class": "Warrior", "level": 11, "loot": 1240},
    {"name": "Tamsin Larkspur", "class": "Mage", "level": 9, "loot": 620},
    {"name": "Wren Hawkridge", "class": "Ranger", "level": 19, "loot": 2480},
    {"name": "Piran Dovecote", "class": "Bard", "level": 4, "loot": 310},
    {"name": "Hild Stonebridge", "class": "Cleric", "level": 13, "loot": 930},
]
```

(a) Loop over the list: each `hero` is a dictionary. Print one line each: name in 20, class in 10, then the loot as `1,240 cr` right-aligned in 9. (Build the money text first, then align it.)

(b) Add up the loot into a total and print it.

(c) Find the most experienced hero. Start with `top = party[0]`, loop, and replace `top` whenever someone has a higher level. Print their name and level.

(d) Build a list of the names of everyone level 10 or higher and print it.

EXPECTED OUTPUT

```
Bram Ironfoot       Warrior    1,240 cr
Tamsin Larkspur     Mage         620 cr
Wren Hawkridge      Ranger     2,480 cr
Piran Dovecote      Bard         310 cr
Hild Stonebridge    Cleric       930 cr
Total loot: 5,580 cr
Highest level: Wren Hawkridge (19)
['Bram Ironfoot', 'Wren Hawkridge', 'Hild Stonebridge']
```

Hint: Inside an f-string, use the other kind of quotes for the key: `f"{hero['name']}"`.

Exercise 5 Counting with a dictionary

10 min · `ex5_class_count.py`

> **RAVEN #405, from Captain Siv, quest master:** *"Heroes per class. In Excel it's a COUNTIF per class, and someone always forgets the new class. I want it to notice new classes by itself. Yes or no: can it?"*

GIVEN

```
classes = ["Warrior", "Ranger", "Mage", "Warrior", "Bard", "Ranger", "Warrior", "Cleric",
"Ranger", "Warrior"]
```

(a) Start with an empty dictionary `headcount = {}`. Loop over the classes: if the class is already a key, add 1; otherwise create it with 1. Print the dictionary.

(b) Print the counts as a table, classes in alphabetical order (10 characters for the name). `sorted(headcount)` sorts the keys.

(c) Find the biggest class and print it with its count.

EXPECTED OUTPUT

```
{'Warrior': 4, 'Ranger': 3, 'Mage': 1, 'Bard': 1, 'Cleric': 1}
Bard      1
Cleric    1
Mage      1
Ranger    3
Warrior   4
Biggest class: Warrior (4 heroes)
```

Hint: This "if it's there add, otherwise create" loop is the most common five lines in all of data work. You'll write it a hundred times, until pandas does it for you in one. For the biggest: start with `biggest = None` and replace it when it's `None` or a count is bigger.

Exercise 6 XLOOKUP, the Python way

10 min · `ex6_lookup.py`

> **RAVEN #406, from Quartermistress Pell:** *"Half my spreadsheets are XLOOKUPs: level to rank, region to warden. Tell me a dictionary is an XLOOKUP and I'll stop being scared of it."*

GIVEN

```
rank_from = {"Novice": 1, "Journeyman": 5, "Veteran": 10, "Champion": 16}
region_warden = {"Greyfen": "Alder Pike", "Ironhills": "Hild Stonebridge", "Northern Wilds":
"Corwin Northwind"}
```

(a) Look up the level a Veteran starts at and the warden of the Ironhills. Then look up Thornwood's warden with `.get` and the fallback `"no warden yet"`. Then print whether `"Greyfen"` is a key.

(b) Print the keys of `region_warden` as a list, and the values of `rank_from` as a list.

(c) Loop with `for rank, level in rank_from.items():` and print one line per rank.

(d) Given `level = 12`, work out the rank: loop over the items, and whenever the level is at least the start, remember that rank. Print the result.

EXPECTED OUTPUT

```
10
Hild Stonebridge
no warden yet
True
['Greyfen', 'Ironhills', 'Northern Wilds']
[1, 5, 10, 16]
Novice from level 1
Journeyman from level 5
Veteran from level 10
Champion from level 16
A level 12 hero is a Veteran
```

Exercise 7 Crowns per day, by monster STRETCH

15 min · `ex7_crowns_per_day.py`

> **RAVEN #407, from Queen Aldara:** *"Everyone wants the Wyrm notices. Are they actually worth it, per day on the road? For the Privy Council. By Thursday. (Which Thursday? A Thursday.) A."*

GIVEN

```
quests = [
    {"monster": "Trolls", "paid": 1320, "days": 9},
    {"monster": "Wyrm", "paid": 0, "days": 14},
    {"monster": "Goblins", "paid": 260, "days": 4},
    {"monster": "Trolls", "paid": 1150, "days": 8},
    {"monster": "Wyrm", "paid": 3420, "days": 16},
    {"monster": "Goblins", "paid": 310, "days": 6},
    {"monster": "Wyrm", "paid": 950, "days": 12},
    {"monster": "Trolls", "paid": 1480, "days": 11},
]
```

(a) Write two functions: `average(numbers)` and `per_day(paid, days)` (crowns per day, rounded to 1 decimal).

(b) Build a dictionary `by_monster` whose keys are monsters and whose values are **lists** of crowns per day. The counting loop from exercise 5, but appending instead of adding 1. Print it.

(c) For each monster in alphabetical order, print the name in 8, the number of quests, and the average crowns per day (`:>6.1f`). Then print the best value.

EXPECTED OUTPUT

```
{'Trolls': [146.7, 143.8, 134.5], 'Wyrm': [0.0, 213.8, 79.2], 'Goblins': [65.0, 51.7]}
Goblins 2 quests    58.4 cr a day
Trolls  3 quests   141.7 cr a day
Wyrm    3 quests    97.7 cr a day
Best value: Trolls
```

Hint: A dictionary whose values are lists is a dictionary like any other. `by_monster[m].append(x)` reads as: find the list for this monster, then append to it. A Wyrm that wins pays a fortune; a Wyrm that wins nothing pays nothing for fourteen days.

Bug hunt Pages from the Grimoire, page 4

15 min · `bughunt/roster_bug.py`

Dalvric's guild roster, last run the week he sailed west. It named the wrong hero as the most experienced, crashed looking up a region that is definitely on the map, and reported one hero in every class. There are **three** bugs: one crashes (read the error twice; the clue is in the quotes), one picks the wrong hero silently, and one makes every count come out as 1.

EXPECTED OUTPUT ONCE FIXED

```
Highest level: Wren Hawkridge
Ironhills: Hild Stonebridge
{'Warrior': 2, 'Ranger': 2, 'Mage': 1, 'Bard': 1}
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.